Youtube:-

Salesforce Complete Course → 1. https://www.youtube.com/watch?v=qcTDDMYWm8E

2. https://www.youtube.com/watch?v=MoCLFftP5GE&list=PLEKSN4V4WEnJi-TCknopE02NbJoP5I8jf&index=22

Apex →

Apex Triggers → https://www.youtube.com/watch?v=xsi06y7NOLA

Apex Flows → https://www.youtube.com/watch?v=KVs_omNlaxg

LWC → https://www.youtube.com/watch?v=bLyAsIeDZtw

SOQL → https://www.youtube.com/watch?v=kw6SFQWQ11Q

Integration → https://www.youtube.com/watch?v=2myol9hI28c

Project →

Omnistudio →

Udemy:-

LWC → https://capgemini.udemy.com/course/zero-to-hero-in-lightning-web-components/

Apex → https://capgemini.udemy.com/course/salesforce-development-course-for-beginners/

Salesforce development → https://capgemini.udemy.com/course/salesforce-development/

Admin + Developer → https://capgemini.udemy.com/course/learn-salesforce-admin-developer-with-lwc-live-project/

Salesforce Project → https://capgemini.udemy.com/course/salesforce-development-project/

What is Salesforce?

Salesforce is a cloud-based Customer Relationship Management(CRM) and application development platform that enables organizations to manage their entire customer lifecycle – sales, service, marketing, operations and analytics- using a single, unified system. It provides business with a secure, scalable and highly customisable environment where they can store customer data, automate processes, build custom applications, integrate external systems, and deliver personalized experiences across every customer interaction

Operating entirely on the cloud, salesforce eliminates the need for hardware, installations or complex infrastructure management

In simple terms, Salesforce is not just a CRM- it is a complete digital platform that helps organisations improve efficiency, strengthen customer relationships, and accelerate business growth.

Salesforce is a SAAS(Software as a Service) and PAAS (Platform as a Service)

Multi Tenant Environment – Multiple customers sharing the same underlying salesforce infrastructure such as servers, databases, system resources.

Salesforce is a full ecosystem of clouds that organizations can choose based on their needs.

#Clouds in Saleasforce

1. Sales Cloud
   Sales cloud is salesforce's core CRM solution designed to help sales teams manage every stage of the customer journey. It provides tools for handling Leads, Accounts, Contacts and opportunities, while offering real time pipeline visibility and accurate sales forecasting. With built-in productivity features, Sales cloud helps teams close deals faster and operate in a more organized and efficient manner.
2. Service Cloud- Service cloud focuses on delivering fast and consistent customer support across channels such as phone, email, chat and social media. It provides case management, omni-channel routing, a centralized knowledge base and CTI integrations. With automated workflows and service tools, it helps support teams resolve issues quickly and deliver a superior customer experience.

3. Marketing Cloud – Marketing Cloud is a comprehensive digital marketing platform used to create personalized customer journeys. It supports email, SMS, WhatsApp and multi-channel campaigns along with advanced customer segmentation and real-time personalization. With powerful automation tools, it enables marketers to reach the right audience with the right message at scale.

4. Commerce Cloud- powers both B2B and B2C e-commerce experiences. It helps businesses build and manage their online storefronts, products catalogs, shopping carts and checkout processes. With integrated personalization and AI

driven recommendations, Commerce cloud enables seamless and engaging shopping experiences across web and mobile

5. Experience Cloud- enables organizations to build secure, branded portals for customers , partners and employees. These sites can support self-service, collaboration and information sharing, helping users access what they need without relying on internal teams. It is widely used for customer communities, partner portals and knowledge-sharing hubs.

6. Analytics (Tableau CRM/ CRM Analtics)- is sf's advanced analytics platform that enables users to build interactive dashboards, uncover trends and make data-driven decisions. It delievers predictive insights using AI and integrates deeply with salesforce data. With real-time visualizations and actionable intelligence, it transforms raw data into strategic business insights.

7. Financial Services Cloud – is built specifically for banking, wealth management and insurance organizations. It provides a relationships-driven data model that connects clients , households, financial accounts and goals. With specialized workflows and industry components, FSC helps advisors and bankers deliver personalized financial guidance and manage client relationships more effectively

8. Health Cloud – Health cloud is designed for healthcare providers, paryers and life sciences organizations. It centralizes patient information care plans and provider relationships in one place. With tools for care coordination and patient engagement, Health cloud enables healthcare teams to deliver more connected, efficient and personalized care experiences

9. Manufacturing Cloud – aligns sales and operations by providing a unified view of sales agreements, demand forecasts and production plans. It helps manufacturers improve accuracy in planning, manage partner channels and streamline collaboration across teams. By connecting data from the field to the factory, it supports better decision- making and operational efficiency

In production code we cannot write a System.debug () it is not a good practice

In VSCode in command palette → sfdx create project

Command for setting or authorizing a dev hub → sfdx force:auth:web:login -a salesDemo -d in terminal

To Authorize a scratch org – add “hasSampleData”:true in config/project-scratch-def.json file and then run command in terminal
- sfdx force:org:create -a lwcScratchOrgOne -d 30 -f config/project-scratch-def.json -s

To open the scartch org

sfdx force:org:open

Authorize an org is most important step in VS code to connect vs code and salesforce

If it fails then try update cli – sfdx update

Then authorize an org – then choose project default →give alias name as your project name in vs code then enter it will open browser for login

Sfdx commands – this command will print all the commands in salesforce

#Objects

Objects are like tables in databases. Fields are similar to columns of the table. Records are like rows of data inside the table

#Limitations

- Every object has 2 master-detail relationships
- Every object has 40 lookup relationships
- We can create upto 2000 custom objects in SF org
- Each object can have up to 500 custom fields.
- Salesforce lets you track changes up to 20 fields on a single object.
- you can only run 100 SOQL queries or 150 DML operations in one transaction
- Each object in Salesforce can have up to 500 active validation rules.
- You can create up to 25 roll-up summary fields on each object and they only work with master-detail relationships.
- Salesforce allows you to create up to 200 record types per object.

#Governor Limits

are built-in controls that stop any user or process from using too many resources in Salesforce’s shared environment. If you go over these limits, your automation fails. These limits ensure the platform runs efficiently for everyone.

#validation rules

Validation rules in Salesforce are a set of rules that define the criteria that users are required to meet when they are creating or updating records to maintain data quality and consistency. Validation rules ensure that data in Salesforce complies with specific business needs and validation logic.

Validation Rules vs. Required Fields

Why not simply make the field required? Required fields are 'all or nothing'; you must complete the field for the record to be created. Validation rules kick in *only under certain conditions*.

Workflow Rule and a Process Builder in Salesforce

Both the workflow rule and process builder are automation tools that allow businesses to define and automate various business processes and actions; however, they have different functions and capabilities.

Process Builder has more automation features and is more versatile. It lets you set up complicated processes with many criteria, steps, and decisions. With Process Builder, you can automate processes that create, update, or delete related records and trigger flows. It can do more than just design, update, and delete records. It can also launch flows and call custom Apex code, so it's better for more complicated business processes with multiple steps and decision points.

Unlike Process Builder, Workflow rules are more basic and have fewer features. They're mainly for automating simple stuff like sending emails, changing fields, and generating tasks based on specific criteria. Workflow rules can update fields, send emails, and create jobs. They're all about ensuring users are updated or notified when certain conditions are met.

#approval process

An approval process in Salesforce is a workflow automation tool that routes records for approval based on predefined criteria. It allows organizations to define rules and steps for approving records, such as opportunities, leads, or contacts. Designated approvers review the record during an approval process and can approve, reject, or reassign it as needed.

Approval processes can include multiple steps, parallel or sequential approvers, and custom notifications to keep stakeholders informed throughout the process.

#Record Types

Record Types are a way of grouping many records of one *type* for that object. These can be applied to any standard or custom object, and allow you to have a **different page layout**, **fields**, **required fields**, and **picklist values**.

Page Layouts

a customization tool that controls the organization and appearance of fields, buttons, custom links, and related lists on record detail and edit pages.

This includes rearranging fields, adding new sections, and controlling the visibility and editability of fields based on user permissions.

#Security in Salesforce

Salesforce uses object-level, field-level, and record-level security to control data access. Profiles and permission sets control object access, field-level security hides or shows specific fields, and sharing settings manage who can see individual records.

Organization-Wide Defaults (OWD)?

OWDs define the base level of access to records: Private, Public Read Only, Public Read/Write, or Controlled by Parent. These settings work with roles and sharing rules to open access as needed.

#Object level access

It verifies that user has permissions to see objects of that type It can be managed through 2 configurations

1. Profiles : It is a mandatory collection of settings and permissions that defines what a user can do within the platform including access to objects, fields, tabs and apex classes.
   It will control access to object level and field level security, ** Assigning each user a profile is still compulsory. It will gives minimum permissions
2. Permission sets: It is the additional set of user permissions on top of profiles
   User can have multiple permissions sets
   Multiple permission sets that are combined are called permission set groups.

#Field Level Security:

It takes care of permission over fields and can be enforced through profiles and permission sets

Having access to a type of object-level data doesn't guarantee access to fields and records.

Record- level Security

Salesforce lays down five ways to control record-level access. These are:

1. Organization-wide defaults.

   The organization-wide defaults of a Salesforce object decide the visibility and editability of its records regarding the access of users who don't own them

   OWD Private: OWD of an object as private, a user won't be able to see any records other than the one they own

   OWD Public read-only : OWD setting will allow all authorized users to **view the records** – provided they can access the object. However, this setting only makes the records visible, and a user can't edit or delete record access unless they own it.

   OWD Public read-and-write: This setting allows anyone to view or **edit an object's records.**

2. Role hierarchies.

   if 'role A' is above 'roles B,' 'C,' and 'D,' it will automatically access all the data that the lower roles possess. Role hierarchies are also referred to as vertical sharing.

3. Sharing rules.

   Sharing rules allow you to share access to records with other users laterally.

   Ownership-based sharing rules

   This type of sharing rule lets you share records based on their owners. The ownership can be **categorized based on role**, public group, or role-and-subordinate.

   Criteria-based sharing rules

   The criteria-based sharing rules allow you to share records **based on criteria.** This criterion refers to a specific value of a field in a record.

   Guest user sharing rules

   This sharing rule type differs from the others. Guest user sharing rules only allow read-only access to records for guest users. Such users aren't authenticated and don't need login access to the organization

4. Manual sharing.

   it is a way to **grant one-off access. The record owner, roles above the hierarchy, and system administrators can manually share a record**

   ensure that the object's OWD is private or public read-only. Otherwise, it wouldn't be worthwhile. You can manually share a record by clicking **the "Sharing" button** on the record-detail page. While sharing the record manually, you can choose the access level, such as read-only or read-write.

5. Apex sharing.

As a system admin or Salesforce developer, you can write an Apex class code to fulfill such **out-of-the-box needs.** A Salesforce developer can easily write an Apex trigger that shares a record with other users.

types of Sharing Rules are available?

You can use owner-based rules to share records based on record owner, or criteria-based rules to share based on record values. Guest and territory-based rules are also available for specific use cases.

How are Public Groups and Queues Different?

Public Groups and Queues are both ways of organizing and managing records and tasks in Salesforce, but they serve different purposes:

Public Groups are users with shared access to records, files, and other resources in Salesforce. They are primarily used for sharing records, collaborating on projects, and simultaneously assigning tasks to multiple users. Administrators can manually create Public Groups and include individual users and other public groups.

Queues are virtual groups in Salesforce used to manage records and tasks that require a collaborative effort or follow a specific workflow. Unlike public groups, queues do not contain individual users but hold records awaiting action or resolution. Queues can be used for various purposes, such as managing support cases, leads, or service requests. Records in a queue can be assigned to individual users or picked up by members based on predefined criteria, such as workload or expertise.

#What is a Person Account and how does it differ from a Business Account?

A Person Account merges account and contact data into one record, used in B2C scenarios. Business Accounts represent companies and are linked to multiple contacts. Person Accounts need to be enabled separately.

Dashboards

Dashboards are an integral part of Salesforce's data visualization platform. They offer a single view of critical business metrics and information in real-time, allowing you to track KPIs and progress and make informed data-driven decisions.

There are two types of Dashboards:

Static Dashboards: These are dashboards where the components and data are fixed at the time of creation, and they remain the same until manually updated by the dashboard creator. The same data is visible whenever a user accesses the dashboard until any change is made manually.

Dynamic Dashboards: Dynamic dashboards enable us to visualize a set of metrics within an organization. These dashboards are designed to provide security controls for Salesforce dashboards. The content on these dashboards is not fixed and refreshes automatically based on intervals and customer interactions. It allows the users to view the most current data available.

#Bucket Field in Salesforce Reports?

A Bucket Field lets you group report data without needing a formula or creating new fields in the object. For example, you can group Opportunity Amounts into categories like "Small," "Medium," and "Large" without editing the actual records. This is great for quick categorization and analysis when you don't want to change your data model.

#Custom Report Type in Salesforce?

Custom Report Types (CRTs) allow you to build reports that go beyond standard relationships between objects. You can decide which objects to include and which fields are available. You can also define optional object relationships to show or hide records based on whether related data exists. Custom Report Types are useful when standard report types don't give you enough flexibility or when you need to report across custom objects.

LWC – Lightning Web Components

Component Composition

Composition is adding component withing the body of another component. Composition enables you to build complex components from simpler building-block components

How to refer child components name in parent components

1. childComponent → <c-child-component></c-child-component>
   Replace capital letter with small letter and prefixed with hyphen

   #DOM

Document Object Model(DOM) is a programming API for HTML and XML documents. It defines the logical structure of documents and the way a document is accessed and manipulated. Basically, The DOM is a tree structure representation of web page .
Shadow DOM – It is a part of the Web Components standard, which lets developers create isolated components with their own DOM structure, styles and behavior.
It brings encapsulation concept to HTML which enables you to link a hidden separated DOM to an element.
Benefits→ DOM queries, event propagation and CSS rules cannot go to the other side of the shadow boundary, thus creating encapsulation.

Lightning Framework:

The Lightning component framework is a UI framework for developing single page applications for mobile and desktop devices.

Lightning Components using two Programming models

1. Aura Components Model
2. Lightning Web Component Model

LWC-
Lwc is a new programming model for building lightning component. It uses the core concepts of web standards
Benefits:
- Lightweight framework
- Better Performance
- No need to learn different framework to develop application in salesforce
- Interoperability with lightning Aura components
- Better testing using Jest
- Better Security

Coexitence and interoperability
Aura components and Lightning web components can coexist and interoperate, and they share the same high level services:
- Aura components and Lightning web components can coexist on the same page
- Aura components can include Lightning web components but not other way around
- Aura components and Lightning web components share the same base Lightning components. Base Lightning components were already implemented as Lightning web Components

- Aura components and Lightning web components share the same underlying services(Lightning Data Service, User Interface API)

  #Salesforce Developer Experience(DX)
  It is a set of tools that streamline the entire development life cycle. It improves team development and collaboration, facilitates automated testing and continuous integration and makes the release cycle more efficient and agile
  1. Visual Studio Code
  2. Salesforce CLI
  3. Salesforce Extension Pack
     #My Domain
     Having a custom domain is more secure, some salesforce features require it
     #Dev Hub
     We have to Enable Dev Hub to
     1. Create and Manage orgs from the command line
     2. View information about Scratch orgs
     3. Link Namespace orgs

        #DevHub VS Scratch org
        1. Dev Hub – It is the main salesforce org that you will use to create and manage your scratch orgs
        2. Scratch Org- It is a source- driven and disposable deployment of salesforce code and metadata. Scartch orgs are driven by source, sandboxes are copies of production

*Scartch orgs do not replace sandboxes

Lead Object – A potential customer

A Lead represents a potential customer who has shown interest in a product or service but has not yet been qualified as an opportunity.

When Lead is converted it becomes an Opportunity, Account and Contact

Salesforce automatically creates an Account, Contact and Opportunity record for further processing.

App in Salesforce

In Sf , an app is a collection of objects, fields and tabs that help manage business processes efficiently

Apex is strongly typed language – which has strict restrictions over types of values and variables

EX: If there is mismatch between the data types, we may get error or exception.

Access Modifiers

Accessibility Level

Private<Protected<Public<Global

1. Private→ only used within same class – not outside of class
2. Protected → only used in inheritance for that make parent class abstract and using protected we can access data of parent in child class
3. Public → Used locally in current org
4. Global → used to access data in one org to different org with the help of AppExchange

Primitive Data Types
Basic data types which can not be defined in terms of other data types

Date→ Date myDate1 = Date.newInstance(2025, 08, 22);
myDate1.DaysBetween(myDate2) → used to calculate difference between 2 dates
myDate1.isSameDay(myDate2) → used to check birthday's or same dates

Derived/ Structured Data Types
Data types which can be defined in terms of other data types
Ex: Class
In string isBlank and isEmpty behave same the difference between both is whitespace calculate in isEmpty
String ms=' ';
System.debug(String.isEmpty(ms)); → false → calculate whitespaces also
System.debug(String.isBlank(ms)); → true
15 digit and 18 digit Id points to same record
18 digit Id- is just the 15 digit Id with 3 extra characters at the end called a checksum. This checksum encodes the capitalization patterns, making the ID case-insensitive. So even if the case is changed salesforce can still match it to the correct record
Because of this whenever you're sharing data with outside systems like Excel or databases, it's better to use the 18-digit version—it's more reliable and avoids case mismatch issues.

#Safe Navigation Operator
? is safe navigation operator it used to avoid nullpointerexception
Account acc;
String accName= acc?.Name;
System.debug(accName); → it will give null as output - if acc is not null then assign value to accName

#Null Coalescing operator
The null coalescing operator(??) in apex is used when we want to give a default value if something is null
In simple words : "if the first value is there, take it. If not, use this backup value"
It saves us from writing long if-else blocks just to handle nulls

Difference between Null Coalescing and Safe Navigation Operator:
Safe Navigation Operator(?.) → Safely access a field or method on an object that might be null if the object is null it returns null instead of throwing an error
Null Coalescing(??) → Provides a default value when an expression is null. If the value is null, it takes the fallback value instead.

break and continue
Break will break the current and next iteration
Continue will stop only current iteration and continue to next

#Wrapper Class
A wrapper class in apex is a custom class that groups together multiple pieces of data (which may be related or from different sources) into a single object
It can hold:

- SObject records
- Primitive data types
- Complex data types
- UI-related fields
- Why do we need Wrapper class?
  In Apex
- A method can only return one value
- That value can be a primitive (string), collection(List<Account> etc), SObject
- But you cannot directly return multiple unrelated things in one go

- Wrapper class solves this problem
- Very useful in complex integration and LWC requirements

Practical Example:

An LWC that passes an AccountId to Apex

Apex needs to return:

-A list of Contacts(related to Account)

-A list of Opportunities(related to Account)

-A Boolean to indicate something like “show special offer” checkbox

Problem:

If we don’t use a wrapper , apex would need to send 3 separate values back to LWC:

One list for contacts

One list for opportunities

One Boolean flag

That means three separate @AuraEnabled variables or three separate methods

More round trips, messay method signature and harder to maintain

```
public class  WrapperClassExample{
@auraEnabled(cacheable=true)
    public static AccountWrapper wrapperMethod(id AccountId){
        list<contact> contacts=[select id, lastName from Contact where accountId=:Accountid];
        list<Opportunity> opp=[select id, stageName from Opportunity where accountId=:AccountId];
        Boolean offer = opp.size()>2;

        AccountWrapper aw= new AccountWrapper();
        aw.conList= contacts;
        aw.oppList= opp;
        aw.specialOffer= offer;
        system.debug(aw);
        return aw;

    }
    //wrapper class
    public class AccountWrapper{
        @auraEnabled public list<Contact> conList;
        @auraEnabled public list<Opportunity> oppList;
        @auraEnabled public boolean specialOffer;
    }
```

#Transient Keyword

The transient keyword marks a variable as temporary, so it is not saved when the object is converted to a string or serialized. It is used for large or temporary data that does not need to persist beyond the current operation.

- Without transient all variables of an object are stored in memory during serialization
- It can only be used with instance variables inside a class.
- Cannot be used with Methods, Static variables, Local variables inside methods

  # Static variables
  A static variable is a class level variable that is shared among all instances of the class. Unlike instance variables, which are unique to each object, a static variable has a single shared value that persists across all instances of the class. Changes made to the static variable by one instance affect all instances of the class because it is associated with the class itself, not with any specific object.
- Static variables hold the value till the transaction will executed once it is completed it will loose the value
- If you want to store the value permanently create a custom field and store

  #Instance variable :
  An instance variable is a variable that belongs to a specific instance of a class. Each object created from the class has its own separate copy of the instance variable, meaning the values of these variables can differ between objects. Instance variables store data that is unique to each instance of the class.

#Static Method

A static method is a method that belongs to the class itself rather than to any specific instance of the class. It can be called without creating an instance of the class and is typically used for operations that do not require access to instance variables. Static methods can access static variables and other static methods within the same class but cannot access instance variables or instance methods

Final Keyword

In apex the final keyword is used to declare constants or prevent modifications

Final variable once assigned its value cannot be changed

Final class/method : you cannot explicitly declare classes or methods as final because apex classes and methods are final by default- you cannot extend a class unless it's marked virtual or abstract

You cannot override a method unless it's virtual or abstract

Class level constants:

Is declared using “static final” belongs to the class itself, not any object-is shared by all instances of the class- is immutable -once set it cannot be changed – used when the value is universal for all objects of the class.

An instance level constants is declared using “final”(without static)- Belongs to each individual object – can be set only once-typically in the constructor- is immutable for that specific object-used when the value is unique per object but should never change

#Virtual

In apex, a virtual method or class allows subclasses to override the functionality of that method or class. It provides flexibility by enabling derived classes to provide their own specific implementations while still inheriting from the base class.

Virtual method has basic logic that can be overridden by the child class

Abstract does not contain logic, and has to be overridden by the child class

Master Detail Relationship

Closely links objects together such that the master record controls certain behaviors of the detail and subdetail record.

Behaviors of master-detail relationships:

- Deleting a detail record moves it to the Recycle Bin and leaves the master record intact; deleting a master record also deletes related detail and subdetail records. Undeleting a detail record restores it, and undeleting a master record also undeletes related detail and subdetail records. However, if you delete a detail record and later separately delete its master record, you can’t undelete the detail record, as it no longer has a master record to relate to.
- By default, records can’t be reparented in master-detail relationships. Administrators can, however, allow child records in master-detail relationships on custom objects to be reparented to different parent records by selecting the **Allow reparenting** option in the master-detail relationship definition.
- The Owner field on the detail and subdetail records isn’t available and is automatically set to the owner of the master record. Custom objects on the detail side of a master-detail relationship can't have sharing rules, manual sharing, or queues, as these require the Owner field.
- Detail and subdetail records inherit security settings and permissions from the master record. You can’t set permissions on the detail record independently.
- The master-detail relationship field (which is the field linking the objects) is required on the page layout of the detail and subdetail records.

- The master object can be a standard object, such as Account or Opportunity, or a custom object.
- As a best practice, don't exceed 10,000 child records for a master-detail relationship.
- Each custom object can have up to two master-detail relationships and up to 40 total relationships.
- The Related To entry can't be changed after you save the relationship.

** You cannot create a new Master-Detail relationship on an existing custom object if records already exist. You must first create a Lookup relationship, populate the lookup field with data in all records, and then change the relationship type to Master-Detail.

LookUp

Exception:

An exception is an event which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.

```
// Math Exception

Double result = 10/0;

System.debug('Result ='+result);

//DML Exception

Student__c record = new Student__c();

Insert record;

//Null Pointer Exception

String str;

System.debug(str.toLowerCase());
```

Exception Methods

```
Catch(MathException e){

e.getCause()
```

e.getMessage()

e.getLineNumber()

e.getStackTraceString()

e.getTypeName()

}

Permission Set:

A permission set is a collection of settings and permissions that give users access to various tools and functions.

Permission sets extend users functional access without changing their profiles and are the recommended way to manage your user's permissions

How to assign permission sets to any user

1. Search permission sets → new→ Give Name→ save → Assigned Apps→ In assigned apps → Edit→ select app name→ Save→ Manage assignement→ assign user → save
2. Check In 'User' → That user gets permission set access

#Data Management Tools

1. Data Import Wizard
2. Data Loader

Difference between Data Import wizard and Data Loader

Data Import Wizard	Data Loader
1. It is in- browser wizard that imports your organization's data	It is desktop application for bulk imports/exports
2. It can be used upto 50k records	It can be used upto 50 lakh records
3. It worked with specific objects(Accounts/Contacts/Leads/Solutions/Custom Objects). It does not support Opportunities object	It worked with all standard and custom objects
4. It gives no detailed log	It gives detailed log
5. It does not support to scheduled data loads	It can be used to schedule

	regular data loads/nightly imports
6. It does not support delete operations	It supports delete operations
7. During import, duplicate data can be ignored	During imports, duplicate data can not be ignored.

Steps to use Data Import Wizard :

1. Prepare the Data file(.csv) file
2. Launch Data Import Wizard
3. Select the Standard/Custom object in the wizard
4. Select the activity to be performed (insert/update/upsert)
5. Upload .csv file
6. Maps the fields in the data file with the fields in the object
7. Perform the import
8. Monitor the job status (Bulk Data Load Jobs Page).
9. Rework on the errored records if any.

Same steps applied for data loader also 1st we need to download and then perform operation

- While inserting records in data loader id does not required
- For update/upsert/delete operations in data loader id is required
- Difference between Export and Export ALL in data loader
- Export only current records of objects
- Export All exports records from recycle bin also
- Using data loader we can export data from 1 object only→ here we can apply conditions on fields
- Using data import wizard we can export data from multiple objects at time→ we cannot apply conditions on fields

#Email Template

-With the help of visualforce template we cannot send mass emails. It is used to fetch data dynamically and send mails

#Workflow Rules

Workflow rules let you automate standard internal procedures and processes to save time across your org.

A workflow rule is the main container for a set of workflow instructions. These instructions can always be summed up in an if/then statement.

Two Parts:

1.Workflow Rule Criteria

2. Workflow Rule Action

Report:

Report is a collection of information and an organized presentation of data about a particular object (or with its related objects) under certain criteria.

Dashboard:

Dashboard is the graphical representation of the data generated by a report(or multiple reports) at certain point in time. So it's a snapshot of data, to obtain the latest data you need to refresh them either manually or in scheduled manner.

Report Builder:

Report builder is a tool for accessing your data quickly and comprehensively. Use it to set up new reports and edit existing ones.

#SObject

There are 2 types of sobjects are 1. Typed sObject – where data type of object is already known so we can access fields using . operator

Ex: Account acc= new Account()

acc.Name="Test"

2. Generic sObject – where data type of object is not known it will be defined at run time so we can fields using get () and put() methods

Ex: SObject obj = new SObject()

obj.put("Name", "test1")

//Create a new SObject Instance and insert data

SObject record=(SObject)Type.forName('student__c').newInstance();

record.put('Name', 'ABC');

record.put('Roll_Number__c', '47);

record.put('College__c', '10238iuhd');

insert records;

Synchronous transactions in Salesforce execute immediately in a single, blocking thread, requiring user waits but offering instant results and data consistency. Asynchronous transactions execute later in the background when resources become available, providing higher governor limits and better performance for long-running, complex tasks.

#Asynchronous Apex

Asynchronous Apex in Salesforce is used to execute long-running or complex processes in the background without blocking the user interface or violating governor limits. It enhances performance, allows handling large data sets, enables external system integration via callouts, and runs with separate, higher governor limits compared to synchronous transactions

1. Batch Apex:
   - Implement Database.Batchable<SObject> interface.
   - Implement 3 methods from interface Start(), execute(), finish()
   - Start() methods accept "Database.BatchableContext bc " parameter and it return object of Database.QueryLocator
   - Execute() accepts 2 param as "Database.BatchableContext bc " and List<SObject> scope(it comes from start ())
   - And finish() method executes only once
   - Maximum batch size is 2000 minimum is 1 and default is 200
   - For every executes method governor limits reset
   - To execute Batch class in anonymous window we write
   - Id BatchJobId= Database.executeBatch(obj, 500) →obje is the object of batch class
   - To schedule Batch apex
   - String ScheduledId= System.scheduleBatch(obj, 'Batch job Name', 30);

     #Database.QueryLocator –
     Database.QueryLocator is a salesforce system-defined apex type(a class-type object) that lives inside the Database class. It represents a server-side handle to a large SOQL result set, designed specifically for Batch Apex so Salesforce can stream and process millions of records in batches

SO:
It is a type(like List, Account, Map etc)
More specifically, it is nested/inner Apex system class
You cannot construct it
You only receive it from salesforce
Batch Apex Best Practices:

- Minimize number of batches
- Tune queries and reduce processing time
- Use Database.Stateful when you really need shared state
- Chain batches to avoid overlapping schedules
- Ensure batch executes as fast as possible
- Avoid relationship subqueries in QueryLocator
- Requery records inside execute() with FOR UPDATE if locking needed
- Don't invoke batch from triggers unless you control job count
- executeBatch just queues, execution can be delayed
- Must use startTest/stopTest in tests
- Subqueries make job slower
- Methods declared as future can't be called from a batch Apex class

2. Queueable Apex
   Queueable apex is essentially a superset of future methods
   Queueable apex is an asynchronous apex framework that allows developers to execute background jobs in a controlled and flexible manner. It provides more power that future methods by supporting complex data, job tracking and structured execution, making it suitable for enterprise- level asynchronous processing
   Advantages over Future Methods
   - Queueable jobs are similar to future methods in that they're both queued for job execution but they provide you with these additional benefits
   - Getting an ID for your job: When you submit your job by invoking the system. enqueueJob method, the method returns the ID of the new job. This ID corresponds to the ID of the AsyncApexJob record. Use this ID to identify and monitor your job, either through the Salesforce UI (Apex Jobs page), or programmatically by querying your record from AsyncApexJob.
   - **Using non-primitive types**: Your queueable class can contain member variables of non-primitive data types, such as sObjects or custom Apex types. Those objects can be accessed when the job executes.
   - **Chaining jobs**: You can chain one job to another job by starting a second job from a running job. Chaining jobs is useful if your process depends on another process to have run first.

- **How to implement:**
- We need to implement 'Queueable' interface in Apex class.
- Also, we need to implement execute() method of this interface which accepts (QueueableContext context) parameter.
- For execution Id jobId = System.enqueueJob(obj2); → obj2 is the object of queueable class
- We can call one queueable process from another queueable class

Apex Flex Queue:

Apex Flex Queue is a salesforce feature that temporarily holds asynchronous Apex jobs when salesforce cannot execute them immediately
It allows salesforce to manage and prioritize background jobs before they are picked up for execution.
Flex queue acts as a waiting area for async Apex jobs:
A Queueable job goes into flex Queue when:

- Multiple async jobs are submitted at the same time
- Salesforce has reached the maximum number of concurrently running async jobs
- System resources are temporarily constrained
- In these situations, Salesforce:
- Places jobs into Flex Queue
- Executes them later based on availability and priority
- Various status in Batch Jobs:
  1. Holding→ Job is submitted but waiting in the apex flex queue until resources are available to move it to the active queue
  2. Queued → Job is in the Apex Jobs queue and waiting to be executed
  3. Preparing → The start() method has been invoked and salesforce is preparing the job duration depends on data size
  4. Processing → The batch job is actively running and executing records
  5. Aborted → The job was manually stopped by a user
  6. Completed → The batch job finished execution (may include processed and failed records)
  7. Failed → The job stopped due to a system error or unhandled failure.

  ##When to use Iterable in Batch Apex
  Iterable Batch Apex allows us to provide our own dataset to the batch instead of relying on a single SOQL query. If QueryLocator gives fixed records, Iterable gives flexibility to define exactly which data gets processed and how
- Allows building dataset from multiple sources

- Supports complex business-driven datasets
- Preserves custom ordering in Lists
- Can process non-SObject data(AggregateResult, Strings, IDs, etc)
- Enables custom logic before batching begins
- Works with dynamic and conditional data sets
- Gives complete control over iteration and record traversal.

Database.Stateful in Batch Apex

Database.Stateful is a marker interface in Batch Apex that allows salesforce to retain the values of instance variables across multiple execute() call during a batch job.

Normally, each batch chunk runs in a separate transactions, so variables reset every time. When a batch class implements Database.Stateful, salesforce preserves those variable values from one chunk to the next within the same batch execution.

We need Stateful when:

- We want to accumulate values across all chunks
- We want to maintain summary information
- We want to store lists of records processed/ failed
- We want final consolidated output in finish()

3. **Scheduled Apex**

Scheduled Apex provides you an ability to run apex code at specific times or periodically.

You can only have 100 scheduled Apex jobs at one time.

How to implement:

We need to implement Schedulable interface in Apex class.

Also, we need to implement execute() method of this interface.

String cronExp = 0 0 13 * * ? 2024';

S.N.	Name	Values	Special Chars
1	Seconds	0-59	None
2	Minutes	0-59	None
3	Hours	0-23	, - * /
4	Day of Month	1-31	, - * ? / L W

5	Month	1-12 (JAN-DEC)	, - * /
6	Day of Week	1-7 (SUN-SAT)	, - * ? / L #
7	Optional Year	null or 1970-2099	, - * /

Meaning of Special Character:

S. N.	Special Character	Meaning
1	,	Give multiple values. For example, use JAN, MAR, APR to specify more than one month.
2	-	Specifies a range. For example, use JAN-MAR to specify more than one month.
3	*	Specifies all values. For example, if *Month* is specified as *, the job is scheduled for every month.
4	?	Specifies no specific value. This option is only available for *Day_of_month* and *Day_of_week*. It's typically used when specifying a value for one and not the other.
5	/	Specifies <u>increments</u>. The number before the slash specifies when the <u>intervals will begin</u>, and the number after the slash is the <u>interval amount</u>. <u>For example</u>, ➢ if you specify 1/5 for *Day_of_month*, the Apex class runs every fifth day of the month. ➢ if you specify 2/5 for *Day_of_month*, the Apex class runs every sixth day of the month.

6	L	Specifies the end of a range (last). This option is only available for *Day_of_month* and *Day_of_week*. <u>For example</u>, ➢ When used with <u>*Day of month*</u>, L always means the last day of the month, such as January 31, February 29 (for leap years), and so on. ➢ When used with <u>*Day_of_week*</u> by itself, it always means 7 or SAT. ➢ When used with a *Day_of_week* value, it means the last of that type of day in the month. For example, if you specify 2L, you're specifying the last Monday of the month.
7	W	Specifies the nearest weekday (Monday-Friday) of the given day. This option is only available for *Day_of_month*. For example, ➢ if you specify 20W, and the 20th is a Saturday, the class runs on the 19th. ➢ If you specify 1W, and the first is a Saturday, the class doesn't run in the previous month, but on the third, which is the following Monday.
8	#	➢ Specifies the *nth* day of the month, in the format *weekday#day_of_month*. This option is only available for *Day_of_week*.

		➢ The number before the # specifies weekday (SUN-SAT). ➢ The number after the # specifies the day of the month. ➢ For example, specifying 2#1 means the class runs on the first Monday of every month.

Examples:

0 0 13 * * ? Class runs every day at 1 PM.

0 5 * * * ? Class runs every hour at 5 minutes past the hour.

0 0 22 ? * 6L Class runs the last Friday of every month at 10 PM.

0 0 10 ? * MON-FRI Class runs Monday through Friday at 10 AM.

0 0 20 * * ? 2024 Class runs every day at 8 PM during the year 2024.

- To execute scheduled job use String jobId =System.schedule('Sameple job', cronExp, obj);
- To check our schedule job in setup → quick Find- Scheduled Jobs→ we can delete from there also
- Status of jobId → after execution we got jobId copy that jobId → open Resource→ search CronTrigger.obj → open → select some fields and query and add where id='copy job Id'

4. Future Methods

#Flows

Salesforce Flows are automation tools that allow you to collect data, apply logic, and perform actions without writing code. They're used to handle everything from updating records to sending emails and help simplify complex business processes.

** Whenever anything is initiating the flow then the data(records) will be saved in {!$Record} global variable

Rollback Mode allows you to test flows without saving any changes to real data. It's useful for development or testing, especially when the flow performs updates or creates records.

Salesforce flows is a automation tool which will help you to automate business process easily with point nd click and declarative tools without writing single line of code

Types of flows

1. Screen flow
2. Auto- Launch flow
    1. Screen Flow
    - It is run after button click or any action taken
    - While creating screen flow – for creating different pages on that pages take only required fields → Like Account take Name of Account and any fields you want like Number of employees

    2. Auto- Launch flow
        a. Record Triggered flow → It will executes when below operation is executed
    - Creation of record
    - Updation of the Record
    - Deletion of the Record
      Record Triggered flow divided into 2 parts → 1. Fast Field Updates 2. Action on Related Record

        b. Scheduled Triggered flow → A **schedule-triggered flow** in Salesforce is an automation tool that runs automatically in the background at a specified time and frequency to perform actions on a batch of records, without requiring user interaction or a specific record change to initiate it
        c. Platform Event Triggered flow → Users can trigger a Flow when a platform event message is received.
        d. Auto-Launch flow
        e. Record Triggered Orchestration Flow

9. How can we trigger a flow from Apex?
You can start an auto-launched flow from Apex code by creating an instance of the flow and passing in any required variables. You use a map to send values into the flow and then call the start() method. This is useful when you need to mix custom code and declarative automation in the same process.

Triggers in Salesforce

- Apex triggers act like trigger points to perform custom actions before or after any changes made to salesforce records, it could be insertions, updates, or deletions
- You can even run a trigger when a record is restored from the Recycle Bin
- Triggers are available on object level. You can write a trigger for every object

- All triggers are bulk triggers by default, which means they can handle multiple records at a time. Always write triggers keeping this in mind.
- Handle before events, not just after events
- Handle Exceptions and show user relevant error instead of generic errors
- Handle delete and undelete events
- Handle complex custom logic that cannot be easily maintained and managed by automation
- Additional validation checks can be put to use to avoid bad data
- Partial success is allowed in Triggers but not in processes which is more of a All or None framework
- Syntax
  trigger TriggerName on ObjectName(trigger_events){
  code
  }
  Trigger events are comma separated events that need to be invoked through the trigger
  ** Database changes are automatically rollbacked if trigger does not run successfully
  Trigger events
  1. Before Insert – After Insert
  2. Before Update – After Update
  3. Before Delete – After Delete
  4. After Undelete – only for some objects

  Context Variables – come from the system.Trigger class

  Context variables are special system-provided variables available inside a trigger that describe the exact situation in which the trigger is running.

  Context variables give your trigger complete awareness of what's happening and what data it can work with at that moment

- Trigger.IsInsert → True when the trigger is fired because new records are being inserted
- Trigger.IsUpdate→ True when existing records are being updated
- Trigger.IsDelete→ True when records are being deleted
- New
- Old
- Size
- Trigger.IsBefore→ True when the trigger is running before the DML operation occurs

- Trigger.IsAfter→ True when the trigger is running after the DML operation has completed.
- Trigger.IsUndelete→ True when deleted records are being restored from the recycle bin

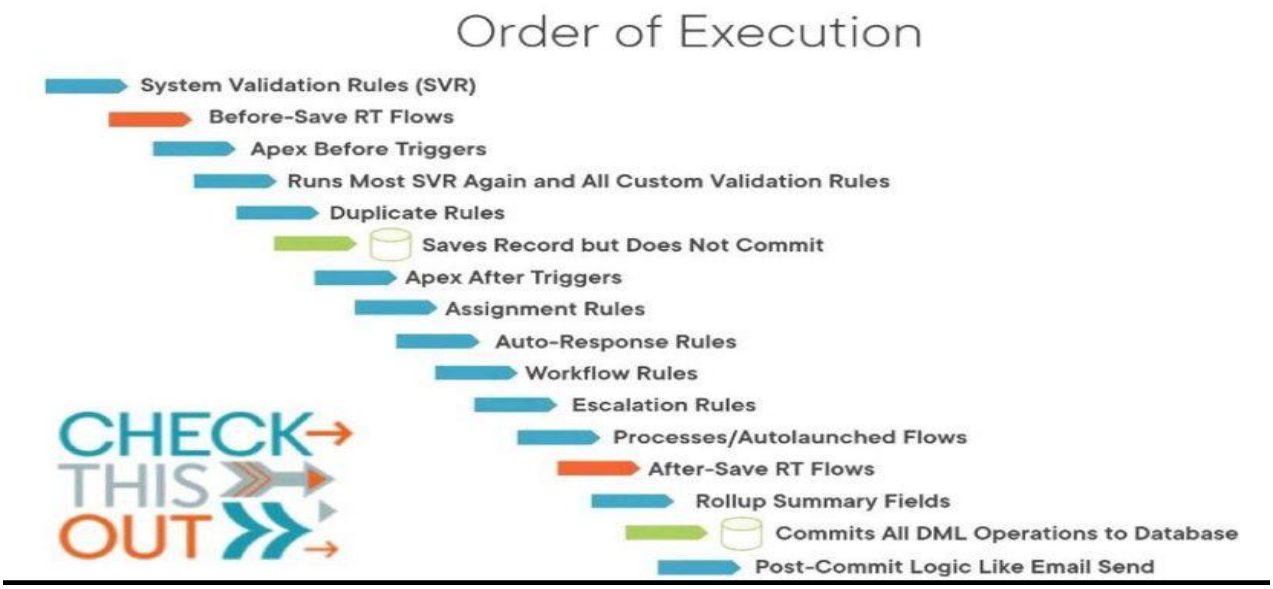
EX: Trigger.isInsert

Trigger.New- Holds the new data inserted or updated

Trigger.OLD – Holds the old data

- You cannot use dml on Trigger.New and Trigger.old variables
- Trigger.OLD is always read only. We can't make any changes on top of it
- We can use trigger.new in before context to update values, but not in after context
- Order of Execution
- 1.Execute Before RECORD Triggered Flows
- 2.Execute ALL Before Triggers- Run system and custom validation rules
- 3.Execute Duplicate Rules – if record is a duplicate no more execution, block the data entry
- 4.Save Record to database, do not commit yet
- 5. Execute all after triggers
- 6. Execute assignment and auto response rules
- 7. Execute workflow rules- if there is field update, run system validations, execute before and after update trigger one more time - deprecated
- 8. Execute Escalation Rules
- 9. Execute processes and flows – no guaranteed order of execution
- 10. Execute record Triggered flows after record save
- 11. Execute Entitlement rules
- 12. If records contains roll up/ cross object reference, parent is updated – same with the grandparent record too and so on
- 13. Execute Criteria based sharing
- 14. Commit to Database
- 15. Execute post commit Logic

Order of Execution After record save



Common Trigger Best Practices:

- One Trigger Per Object

- Use Trigger Handler/Trigger Framework

- Bulkify all Logic

- Avoid Recursion

- Use Context Variable Properly

- Don't do DML or SOQL inside loop

- Use Maps Efficiently

- Use Proper exception Handling

- Have some bypass logic/ Turn off-on Logic

Deactivating Trigger in Production

For deactivating trigger in production we can add custom metadata and with the help of that we deactivate don't directly check an uncheck the isActive checkbox in triggers in production

For this create custom metadata and 2 custom fields trigger name and isActive checkbox and add our trigger to custom metadata types(manage_contact_trigger) and add that settings to our trigger and handler class and run and during production change settings through these custom metadata

```
public class ContactTriggerHandler {
    public static boolean isTriggerActive(String triggerName){
        Contact_Trigger__mdt conSettings= Contact_Trigger__mdt.getInstance(triggerName);
        if(conSettings != null && conSettings.isActive__c== true){
            return true;
        }else{
            return false;
        }
    }
```

In Trigger

```
if(!ContactTriggerHandler.isTriggerActive('ContactTrigger')){
    system.debug('Trigger is not active for this object');
    return;
}
```

***** Tell these as project scenarios**********

Bypass a Trigger for particular User

Using custom permissions- create new custom permissions give name and save→Add these permission to permission set(custom permission) save→ go to user and then permission sets → assign these permission set to enabled permission sets & save

Add logic in handler class using apex class FeatureManagement and method checkPermission

public static boolean hasBypassPermission(String permissionName){

return FeatureManagement.checkPermission(permissionName);

}

In Trigger

If(ContactHandler.hasBypassPermission('Bypass_Contact_Trigger')){

System.debug('Trigger bypass for this user' +UserInfo.getUserName());

Return ;

Operations that do not invoke Triggers:

1. Cascading Operations Don't fire Triggers
2. Bulk Administrative Operations Don't fire Triggers
   - Mass campaign member status updates
   - Mass division transfers
   - Mass address updates
   - Mass approval request user transfers
   - Mass email operations

- Changing a user's default division(with transfer option)

3. Metadata/ Structural Changes Don't Fire Triggers
   - Changing custom field type
   - Renaming picklist values
   - Managing price books
   - Replacing picklists
   - Modifying templates(BransTemplate, MassEmailTemplate, Folder)
4. FeedItem Update Trigger does not fire when like count increases
5. Some updates via Opportunity Product does not fire Opportunity triggers

LWC Lifecycle Hooks

A lifecycle hook is a callback method triggered at a specific phase of a component instance lifecycle

1. Mounting Phase –
   a. constructor()
   - This hook is invoked, when a component instance is created
   - You have to call super() first inside this
   - At this point, the component properties won't be ready yet
   - To access the host element, use this.template
   - This method lifecycle flows from parent to child component
   - We can't access child elements in the component body because they don't exist yet
   - Don't add attributes to the host element in the constructor

   b. connectedCallback()
   - Called when the element is inserted into a document
   - This hook flows from parent to child
   - You can't access child elements in the component body because they don't exist yet
   - You can access the host element with this.template
   - Use it to: perform initialization tasks, such as fetch data, set up caches, or listen for events(such as publish-subscribe events)
   - Do not use: connectedCallback() to change the state of a component, such as loading values or setting properties. Use getters and setters instead.

   c. render() – it is not hook it is a protected method

   d. renderedCallback()
   - Fires when a component rendering is done
   - It can fire more than once
   - This hook flows from child to parent

- When a component re-renders, all the expressions used in the template are re-evaluated
- Do not use renderedCallback()
- To change the state or update property of a component
- Don't update a wire adapter configuration object property in renderedCallback(), as it can result in an infinite loop

2. Unmounting Phase
   disconnectedCallback()
   - Fires when a component is removed from the DOM
   - It flows from parent to child
   - This callback method is specific to Lightning web components, it isn't from the HTML custom elements specification
   - Ex: it is used when we set window.addEventListener('click', this.handleClick) means whenever we click on whole window call handleclick method but due to this memory lick issue happens. To solve this issue we add removeEventListener in disconnectedCallback – due this it will remove window listener when we remove component form page
   - Very useful for memory lick and performance
3. Error Phase
   errorCallback(err, stack)
   - This method called when a descendant component throws an error in one of its callback
   - The error argument is a javascript native error object, and the stack argument is a string
   - This callback method is specific to Lightning web Components, it isn't from the HTML custom elements specification

   #Render Method
   - Render is a method that tells the component which template to load based on some conditions. It always return the template reference
   - The render() method is not technically a lifecycle hook. It is a protected method on the LightningElement class
   - Call this method to update the UI. It may be called before or after connectedCallback()
   - It's rare to call render() in a component. The main use case is to conditionally render a template.
   - renderMethod
   - renderMethod.html
   - renderMethod.js
   - renderMethod.js-meta.xml
   - signinTemplate.html

- signupTemplate.html
- When to prefer multiple template over if:true/ if:false
- If:true/if:false is recommend whenever there is small template to hide and show
- Ideally it's always recommend to break down your component into smallest unit
- Whenever we have a scenario in which we have same business logic but we want to render a component with more than one look and feel
- Whenever we have two design in same component but not want to mix the HTML in one file.

#Components Communication Approaches

1. Parent To Child Communication
2. Child To Parent Communication
3. Sibling Component Communication using PubSub
4. Communication Across VF pages, Aura and LWC using Lightning Messaging Service.
   1. Parent To Child

- To make a field/ property or method public, decorate it with @api decorator
- When we want to expose the property we decorate the field with @api
- An owner component that uses the component in its HTML markup can access the component's public properties via HTML attributes
- Public properties are reactive in nature and if the value of the property changes the component's template re-renders
- EX : @api username
  Call child component in parent html using cabab case and pass data in between child name
  <c-p2c-alert-component
  Message="Hurray !! I got the date"
  ></ c-p2c-alert-component>
  Make a public property in child component to catch data from parent using @api message in .js file also import api from lwc
  To print data call these public property in html in {}

Parent to child communication on action at parent component

- In parent html add child component using cabab case also add some styling
- Catch the value coming from parent in child using @api and in child html file add that property in {}

- We want to pass value that is coming from ui so use it in progress-value={percentage}
- Create one hard coded value in parent js file percentage =10
- To catch that value write in parent html as onkeyup={changeHandler} value={percentage}
- In parent js file write method changeHandler(event){ this.percentage = event.target.value}

Calling child method from parent component
Create one child component
Create one method call it in child html then provide description of that method in child js file using @api
For calling this child method in parent .js write one method and in that this.template.querySelector('c-p2c-slider-component').resetSlider()

Child to Parent Communication

1. Calling parent using simple Event
2. Calling Parent using Event with data
3. Event Bubbling
   Create and Dispatch Events
   *CustomEvent Syntax

new CustomEvent('event name', {bubble:false, composed:false, details:null})

Ex: new CustomEvent('show', {bubbles:true, details:'hello'})

Event name is defined as "show"

By default bubble and composed is false but explicitly we have made bubbles true for event bubbling

Details is used to pass data with event and here I am pass "hello"

- DispatchEvent syntax
  EventTarget.dispatchEvent(eventcreated)
  this.dispatchEvent(new CustomEvent('show',{bubbles:true, details:'hello'}))
  'this' is a eventTarget and dispatchEvent trigger the customEvent
  Event Name Naming Conventions- only Strings, no uppercase letters, no spaces, use underscores to separate words

  Event Bubbling – floating of event from bottom to top
  Lightning web component has only two event propagation
  bubbles – A Boolean value indicating whether the event bubbles up through the DOM or not. Defaults to false

composed – A Boolean value indicating whether the event can pass through the shadow boundary defaults to false

#Event Bubbling Demo
  // 3rd onshow will capture the event based on the eventname and parentHandler will triggered
  // 2nd Y will triggered and then bubbled
<button onclick={X}>Click me</button> // 1st X will triggered and then bubbled

this.dispatchEvent(new CustomEvent('show', {bubbles:true, details:'hello'}))

#PubSub Model – publisher-subscriber model
Component communication using pubsub
There are two ways to communicate between Independent Components

1. Pubsub
2. Lightning Messaging service
   Use this approach, if Lightning Messaging service not serve your purpose.
   It's an old technique to communicate with the independent components in LWC
   Service component don't need html file it will only requires javascript file

Lightning Message service

LMS is the first salesforce technology which enables you to communicate between visualforce, Aura component and Lightning web component anywhere in lightning pages including utility components

In winter 20, Salesforce is released "LMS".

Prerequisite to Create the Message Channel

1. Create a folder called messageChannels under the path force-app\main\default
2. Create a file CHANNELNAME.messageChannel-meta.xml inside the folder
3. Add following XML definition to message channel file
   ```
   <?xml version="1.0" encoding="UTF-8" ?>
   <LightningMessageChannel
   xmlns="http://soap.sforce.com/2006/04/metadata">
     <masterLabel>SampleMessageChannel</masterLabel>
   ```

```
  <isExposed>true</isExposed>
  <description>Message Channel to pass data</description>
  <lightningMessageFields>
    <fieldName>lmsData</fieldName>
    <description>This is the field to pass data</description>
  </lightningMessageFields>
</LightningMessageChannel>
```

4. Update the manifest/package.xml file by adding the LightningMessageChannel

```
<types>
        <members>*</members>
        <name>LightningMessageChannel</name>
</types>
```

5. API Version should be above 47 & above

LWC TO LWC Communication using LMS

1. Reference a message channel in LWC
   Import SAMPLEMC from
   "@salesforce/messageChannel/SampleMessageChannel__c"
2. Import LMS API
   Import {APPLICATION_SCOPE, publish, subscribe, unsubscribe, MessageContext } from 'lightning/messageService';
3. MessageContext wire adapter is use to get information of all LWC using LMS
   @wire(MessageContext) context;
4. Publish a message
   //publish(messageContext, messageChannel, message)
   Publish(this.context, SAMPLEMC, message)
   //this.context is the MessageContext object
   //SAMPLEMC is the reference of the Message Channel
   //message is the data to publish

5. Subscribe a Message channel to receive the message

```
//subscribe(messageContext, messageChannel, listener, subscriberOptions)
this.subscription = subscribe(this.context, SAMPLEMC, (message)=> {this.handleMessage(message)}, { scope: APPLICATION_SCOPE})
//this.context is the context of the current lightning web component using LMS
//SAMPLEMC is the reference of the Message Channel
```

//listener is a function to receive messages on a message channel from anywhere in the application, pass the subscriberOptions parameter as {scope: APPLICATION_SCOPE}

6. Unsubscribe to a Message channel
   unsubscribe(this.subscription)
   this.subscription = null

   LWC To AURA Communication using LMS
   1. Add lightning:messageChannel component to AURA Component
      <lightning:messageChannel type

SOQL

LIKE

LIKE keyword provides a mechanism for matching partial text strings

It supports two wildcards % and _

% stands for occurrence of zero or more characters

_ stands for occurrence of exactly one character.

OFFSET

Offset is used to skip top records

Ex: OFFSET 2 → It will skip top 2 records and print from 3rd record

Insert records in database

insert records; // partial success is not allowed – if 1 or 2 records fail it will rollback a whole transaction

Database.insert(records, true); // partial success is not allowed

Database.insert(records, false); // partial success is allowed – if 1 or 2 records fail it will insert remaining successful records.

Undelete

It will restore deleted records.

List<College__c> deletedRecords =[SELECT Id, Name, College_ID__c, City__c FROM College__c WHERE isDeleted=true ALL Rows];

Undelete deletedRecords;

Aggregate Functions

When we use aggregate function in soql query then it will return array of aggregateResults []

AggregateResult[] records=[SELECT AVG(Amount) FROM Opportunity];

Database.Query()→ Is a method in the Apex Database class that lets you execute a SOQL query that's written as a string at runtime. It is the gateway to run Dynamic SOQL queries in Apex

Return type is always List<SObject> you want typed data, you need to cast it.(List<Account>)

Need of Database.query():

- Query structure is not known until runtime(object name, fields come from user/config)
- You're building a reusable or generic component that can work for multiple objects
- You want to make your code flexible, not tied to a single soql statement

**SOQL Injection→

It is occur due to dynamic soql hacker will pass wrong object name and take or steal the data

SOQL injection happens when a query dynamically includes untrusted user input, allowing attackers to manipulate it and access or modify data they shouldn't

Ways to prevent SOQL injection:

-use static SOQL with bind variables

-Validate user input

-Use String.escapeSingleQuotes()

#ALL ROWS

With the help of this row we can get deleted rows also

EX: list<Account> accList = [select id, name from account ALL ROWS

#For Update

It is used to lock the records where we are performing the transaction so that no other user can modify this records

But it is not used globally as it is not efficient for large data sets or records

#SOQL Best Practices:

- Avoid SOQL Inside loops

- Use only required fields

- Use where Clause to limit the results

- Use SOQL for loop for large data

- Use Nested Queries only when absolutely needed

- Avoid deep nested queries

Use Bind Variables

Avoid Dynamic queries if possible

Use EscapeSingleQuotes() to avoid SOQL injection

#SOSL

SOSL(Salesforce Object Search Language) allows you to perform a text-based search, enabling you to find data across multiple objects and fields in a single query.

*Key Points-

- Used to search text, email and phone fields across multiple objects at once

- Returns data as a list of lists of sObject(one list per object searched)

Can search multiple objects and multiple fields in a single query

Results can be filtered, limited and ordered inside the RETURNING clause

Commonly used for global search, lookup-like functionalities and search bars in custom UIs

Does not support aggregation(COUNT, GROUP BY) or numeric/ data searches.

EX: FIND ‘edge’ IN ALL FIELDS RETURNING Account(id, name), contact(id, name)

Using bind variable

String inputVar= ‘edge’

List<List<sObject>> resultList = [FIND :inputVar IN ALL FIELDS RETURNING Account(Id, name), contact(id, name)]

Dynamic SOSL :

Dynamic SOSL refers to the creation of a sosl string at runtime with apex code. Dynamic SOSL enables you to create more flexible applications. For Ex: you can create a search based on input from an end user, or update records with varying field names

To create a dynamic SOSL query at run time, use the search query method for Ex;

List<List<SObject>> myQuery = search.query(SOSL_search_string);

String searchQuery= 'FIND \'Edge*\' IN ALL FIELDS RETURNING Account(id, name), contact, opportunity

List<List<SObject>> resultList = search.query(searchQuery);

DML → Data Manipulation Language

Dml in sf is a set of Apex statements used to insert, update, delete or restore records in the database.

It allows developers to modify data stored in salesforce objects programmatically.

DML stands for Data Manipulation Language

It is used in Apex to make changes to data stored in salesforce database

DML allows developers to insert, update, upsert, delete or undelete records

It is the write counterpart of SOQL/SOSL which are used for read operations

Without DML, records created in Apex exist only in memory and are not saved to salesforce

DML statements commit changes to the database as part of a single transaction

Salesforce enforce governor limits on DML- a maximum of 150 DML statements per transaction.

Upsert Operation in DML

Upsert is a DML operation in salesforce that inserts a record if it does not exist and updates it if it already exists(based on the record's Id). It combines the functionality of both insert and update into a single statement

It works for both

Counts as 1 DML operation , no matter how many records are inserted or updated

Helps avoid writing separate logic for insert and update

Can improve performance in scenarios where you're unsure whether records already exist

Supports partial success handling using Database.upsert() with allOrNone=false

#Apex Unit Test/ Test Classes:

Apex unit tests are specially written apex programs designed to verify that your business logic, triggers and apex code behave correctly in salesforce. They run in an isolated test environment , simulate real execution without affecting actual data and help ensure that code is reliable, scalable and safe before it is deployed to production

Key Points

- Verifies that apex code performs the intended business logic
- Executes in a dedicated test context without impacting real data
- Ensures system stability and prevents production failures
- Mandatory requirements for deployment to production
- Must achieve minimum 75% code coverage across the org
- Helps identify bugs, edge cases and negative scenarios early
- Builds confidence in functionality, performance and maintainability
  Test classes are essential to ensure that apex code runs correctly, safely and reliably before it reaches production. They help validate that business logic works as expected, prevent unexpected failures and protect both the organization and the salesforce platform from bad code, performance issues and incorrect data processing
  WHY 75% Coverage?
  SF choose 75% code coverage as a practical balance between ensuring quality and maintain development flexibility. The goal is not to force unrealistically high testing requirements that slow delivery, but to guarantee that most of the codebase is validated, stable, and safe to run in a multi-tenant environment, A threshold like 85-90% would be excessively restrictive for large enterprise systems with complex logic, whereas 75% provides strong assurance of reliability while still allowing room for edge cases , framework code, or specific scenarios that may not require explicit testing.

Annotations in Test Classes:

Annotations in Apex test classes are special keywords placed above classes or methods to modify how salesforce treats them during test execution. They help salesforce understand which code is test code, how test data should behave, scope and execution rules of test classes and test methods

@isTest(class Level)

@isTest(Method level)

@isTest(SeeAllData=true/false) → bydefault is false . It allows the test to access real salesforce org data instead of only using test-created data.

@isTest(IsParallel=true)→ Executes test methods in parallel instead of sequentially, reducing total execution time for large test suites

- Applies only at the class level(not allowed on methods)
- Enables parallel execution of test methods in the class
- Default behavior(without this annotation)= sequential execution but order is not guaranteed, only one runs at a time)
- Improves test execution speed, especially in large orgs
- Salesforce runs tests in parallel only when resources allow
- Do not use when tests depend on shared records or modify same data
- Cannot be used with: @isTest(SeeAllData= true) on the same test method
- If you need seeAllData=true, keep that method in a separate test class

@TestSetup→ it is a special method annotation used to create common reusable test data for all test methods in a test class.

- Runs once per test class, before any test method executes
- Data created is available to every test method in that class
- Helps avoid duplicates test data creation
- Improves test execution speed and performance
- Ensures consistent test data across methods
- You can have only one @testSetup method per class.

@TestVisible→ used to expose private or protected variables and methods to test classes without making them public. It temporarily provides broader access only during test execution, without changing visibility for normal runtime code. This means you don't need to make internal logic public hust to test it- encapsulation remains intact in production

- Make private/ protected members accessible only to test classes
- Support access to Methods, member variables, Inner classes
- Visibility change applies only for testing, not runtime execution
- Avoid overuse prefer testing public behavior first

#Test.StartTest() and Test.StopTest()

Define the part of a test method where salesforce actually evaluates the code under test, separating setup logic from execution, resetting governor limits and ensuring asynchronous code runs

- Test.startTest()
- Marks the beginning of the actual execution phase of the test
- Resets governor limits for the code execution after it
- Used to isolate the logic under test from setup logic
- Test.stopTest()

- Marks the end of the execution phase
- Forces asynchronous jobs(future, queueable, batch) to complete
- Captures final governor limit usage for assertions

When to use it

1. When Testing Asynchronous Apex-
   This is the most important and non-negotiable case
   Use startTest() and stopTest() when your code uses- @future, Queueable, Batch Apex, Scheduled Apex
   Why?
   Async code does not execute immediately
   Test.stopTest() forces salesforce to run it during the test
2. When we want to Reset Governor Limits
   Use them when:
   Your test creates a lot of data
   Multiple triggers or flows fire
   You are close to governor limits
   Why?
   Setup logic may consumes limits
   startTest() resets limits before the actual logic runs

@isTest(OnInstall=true)(Manages packages/ AppExchange)→ used in managed packages to automatically run specific test methods when a package is installed from the AppExchange or upgraded in a subscriber org. It ensures that the package works correctly in the customer's environment immediately after installation

Only applicable to test methods

@isTest(seeAllData= true)

@isTest(seeAllData= true) is a test class or test method setting that allows the test to access real salesforce org data instead of only using test-created data. When this flag is enabled, the test can read existing records like Accounts, Contacts, Users, Custom Settings and Custom Metadata from the organization

By default, tests run with seeAllData= false(recommended)

With true, the test can see and use actual org data

Useful for:

- Legacy orgs without proper test data creation
- Testing functionality dependent on existing configuration data
- Scenarios involving Standard Price Book, Custom settings, Metadata

- Not recommended for routine testing because:
- Make tests dependent on org data
- Tests may fail if real data changes
- Reduces reliability and portability

#Test Data Factory

A Test Data Factory is a reusable utility class that centralizes the creation of test data, so test classes remain clean, consistent, and focused on validating behavior rather than setting up data

- Prevent duplicate data setup code across test classes
- Keeps test methods clean and readable
- Makes test maintenance easy when data requirements change
- Encourages consistent and predictable test data
- Helps tests focus on logic and assertions, not setup
- Works well with @testSetup for shared test data
- Supports creation of valid, invalid and edge-case data
- Improves test performance in large test suites
- Should contain only data creation logic(no assertions)
- Makes test code look professional and production-ready

#Test Classes Best practices

- Test behaviour, not just code coverage
- One test method should test one scenario
- Always use assertions, coverage alone is not enough
- Create your own test data, avoid seeAllData=true
- Use a test data factory to avoid duplication
- Use Test.startTest()/ Test.stopTest() only when needed
- Keep test methods independent of each other
- Test both positive and negative scenarios
- Avoid hardcoded IDs
- Test for bulk operations, not single records only
- Keeps tests lightweight and fast
- Use meaningful test method names
- Do not write business logic inside test classes
- Use @testSetup for common test data when appropriate
- Write tests assuming real production conditions

##Integration

Integration is connecting two/more systems to communicate for data transfer or for a process to be done

Integration is the structured and secure exchange of data and functionality between independent systems so that they can work together as a single business process, without being tightly coupled

-Enables communication between independent systems

- Allows controlled exchange of data and functionality

- Preserves autonomy of each system

- Operates through well-defined contracts

- Supports real-time and asynchronous interactions

- Reduce manual intervention and data duplication

- Enables end-to-end business processes across systems

- Promotes loose coupling and long-term scalability

- Improves consistency and reliability of enterprise data

An integration can be classified based on:

- Who initiates the communication
- How data flows between systems
- When the communication happens
- How systems interact
- Which API or technology is used

#API

In Technical terms, API means its Application programming interface/piece of code, which takes request from external system and sends acknowledge/ response

API is basically point of contact for System A to communicate to System B

API is a contract that allows one software system to securely request data or functionality from another system without knowing its internal implementation

- An API defines what can be accessed, how it can be accessed and under what rules enabling systems to communicate in a controlled and reliable way
- APIs act as a communication contract between systems
- They expose capabilities, not internal logic
- They allow systems to remain independent and loosely coupled
- They define security rules through authentication and authorization
- APIs are the foundation on which integration are built

#Outbound Integration By Callouts

When salesforce wants data from external system

Outbound integration is when salesforce initiates the communication and calls an external system. In this case, salesforce acts as the caller and sends data or requests outside salesforce

#Inbound Integration

When any external system sending data to salesforce

Inbound integration is when an external system initiates the communication and calls salesforce. In this case, salesforce acts as the receiver and accepts data or requests from outside systems

This classification is always defined from salesforce's perspective and helps us understand whether salesforce is acting as a caller or a receiver in the integration

One-Directional Integration: A one-directional integration is when data flows in only one direction between systems

One system acts as the source of truth, and the other system only receives data. There is no synchronization back to the source system

Bi-Directional Integration :- A bi-directional integration is when data flows in both directions between systems. Updates made in one system are reflected in the other system and vice versa. This type integration requires data synchronization and conflict handling

Synchronous Integration:- A synchronous integration is when communication happens in real time. The calling system waits for a response before continuing further processing

The success or failures of the integration is known immediately, which can directly impact user experience and response time.

Asynchronous Integration:- An asynchronous integration is when communication happens in the background. The calling system does not wait for the processing to complete

This approach is commonly used for long-running, high-volume or non-critical operations where immediate feedback is not required

This classification helps decide how an integration should be designed to balance responsiveness, performance and scalability.

#Things required for callouts

1. Authorization
2. API Details

- Endpoint URL
- GET/POST/PUT/PATCH/DELETE
- Request Body
- Response

3. Add External system to Named credentials/Remote Site Settings

For any Callouts below code is compulsory

```
Http http = new Http();
HttpRequest req = new HttpRequest();

req.setEndpoint('https://abc.com/animals'); //Endpoint
req.setMethod('POST'); //Request Method
req.setHeader('Content-type', 'application/json;charset=UTF-8');
req.setBody('{"name":"mighty moose"}'); //Request Body

HttpResponse res = http.send(req);
```

An Integration is composed of the following building blocks:

- Participate systems
- APIs
- Authentication and authorization
- Data formats
- Communication patterns
- Timing model
- Error handling and reliability
- Business orchestration logic

SOAP was developed as an intermediate language so that applications built on various programming languages could talk easily to each other and avoid the extreme development effort

SOAP API provides a powerful, convenient and simple SOAP based web services interface for interacting with salesforce

You can use SOAP API to create, retrieve, update or delete records

You can also use SOAP API to perform searches and much more

#Project

Car Hub Project

Setup → App manager → create new lightning app → Name it add color logo next add accounts object, add system administrator → Create one custom object as car → add fields in the car object using fields & Relationship → create a different custom fields needed → create static resources → Setup→ Static Resources

Upload data in csv in cars object → import → cars object select→ csv select→ choose file import → Next it will import data and map

For creating custom layout → 1st check in Lightning app builder if not present then create using custom

For that we creates Page Template in vs code refer that so it will give us 3 columns page layout → name (label) given in .design class will display in lightning app builder

Select that page layout → give a name → it will come to car hub page

Create 3 lwc component add some layout in meta xml file → add these component on car explorer page

CAR_HUB Project Learnings

- Project Setup and org setup
- App Creation and setting up the branding
- Object and fields Creation
- Lightning Page Creation
- Static Resources and Data loading
- Custom Page Layout Designing
- Different form elements and filtering
- Lightning Message Service
- Wire adapter and functions calling
- Imperative Apex calling
- Parent to child and child to parent data passing
- Lifecycle hooks and Navigation
- getRecord, getObjectInfo and getPicklistValues adapter
- LDS(lightning-record-view-form)
- Creating placeholder component
- Styling, looping, properties handling